

Grupos

- o trabalho pode ser realizado em grupos de até 3 alunos

O que entregar

- código fonte (comentado) na linguagem de programação de sua preferência
 - + instruções de compilação/interpretação e execução código
- um arquivo de exemplo do *FURGfs4* com dados dentro
 - colocar os dados do grupo (nome, matrícula e e-mail) dentro de um arquivo chamado *autores.txt* armazenado na raiz do próprio *FURGfs4*

Quando entregar e Onde entregar

- tarefa no AVA até 14/09/2025 23h55

Peso: 50% da N3 (3º bimestre)

Sistema

- o Sistema Operacional alvo para o qual a aplicação será escrita é de livre escolha
 - e não fará diferença neste trabalho

Apresentação/Demonstração em aula

- os alunos devem ser capazes de demonstrar o funcionamento do seu programa em sala de aula
- todos os alunos estar aptos a apresentar qualquer parte do trabalho

Atenção

- todas as entregas serão submetidas a verificação de plágio por “medida de similaridade de código”

O *FURGfs4* é um pequeno sistema de arquivos que reside inteiramente dentro de um outro arquivo a ser armazenado em um sistema de arquivos real. Ele usa conceitos vistos em aula como FAT (tabela de alocação de arquivos) e operações sobre **arquivos** e **diretórios**. O *FURGfs4* suporta diretórios e um número finito (> 100) de arquivos. Sua tarefa é criar um programa que implementa o *FURGfs4*. As operações suportadas são:

1. criar um *FURGfs4* no tamanho escolhido pelo usuário (você pode delimitar opções tamanhos mínimos e máximos, desde que mínimo $\neq$ máximo)
 - esta opção resultará na criação de um novo arquivo no sistema de arquivos real (e.g., trabalhos/S0/furgfs3.fs)
 - este novo arquivo terá o tamanho escolhido para o *FURGfs4*
 - i.e., se o usuário escolheu criar um sistema de arquivos de 800MB, o arquivo furgfs3.fs terá 800MB.

TRABALHO III – ESPECIFICAÇÃO – FURGfs4

2. copiar um arquivo de um sistema de arquivos real (disco, pendrive, etc) para dentro do *FURGfs4* (`cp <origem>/arquivo <furgfs>/arquivo`)
3. copiar um arquivo de dentro do *FURGfs4* para um sistema de arquivos real (disco, pendrive, etc) (`cp <furgfs>/arquivo <destino>/arquivo`)
4. renomear um arquivo armazenado no *FURGfs4* (`mv antigo.txt novo.txt`)
5. remover um arquivo armazenado no *FURGfs4* (`rm arquivo.mp3`)
 - `rm -r` é extra
6. listar os arquivos/diretórios armazenados no *FURGfs4* (`ls`)
 - mostrar tamanho real do arquivo e tamanho ocupado no *FURGfs4*
7. listar o espaço livre em relação ao total do *FURGfs4* (`df`)
 - e.g., 301MB livres de 800MB
8. proteger/desproteger arquivos de escrita/remoção (`protect`)
9. modo debug para listar o índice dos blocos físicos que compõe o arquivo (`debug arquivo.pdf`)

- alguns parâmetros devem ser decididos na implementação:
 - tamanho do bloco
 - tamanho máximo do nome dos arquivos
 - ordem que as partes internas são armazenadas (com exceção do cabeçalho)
 - metadados (obrigatório apenas nome e tamanho do arquivo)
- o tamanho do sistema de arquivos é de escolha do usuário mas não pode ser muito pequeno a ponto de o sistema de arquivos não ser utilizável
- os primeiros *bytes* devem conter um cabeçalho contendo informações necessárias para encontrar o restante das informações
 - tamanho do cabeçalho
 - tamanho do bloco
 - tamanho do sistema de arquivos
 - endereço de início da FAT
 - endereço de início do diretório raiz
 - endereço de início dos dados

- O trabalho terá **requisitos** mínimos e recomendados
 - trabalhos que implementem apenas os requisitos mínimos terão nota oscilando próximo à média (7)
 - para alcançar notas maiores será necessário implementar (parte) dos requisitos recomendados
 - para trabalhos desenvolvidos em grupo espera-se mais funcionalidades que aqueles feitos de forma individual

- não carregar o sistema de arquivos inteiro na RAM, carregue apenas o que é necessário para completar a operação
- dica
 - usem rotinas de (de)serialização para armazenar estruturas de dados
 - entradas de diretório ocupando exatamente o tamanho de um bloco
- exemplos de funcionalidades recomendadas (extra)
 - buscar arquivo em todo sistema de arquivos
 - desfragmentação
 - usar compressão ao armazenar os blocos de dados no *FURGFs*
 - metadados adicionais
 - data de criação e/ou modificação
 - quota por diretório
 - mostrar sha256sum p/ auxiliar na verificação de integridade dos arquivos
 - comparar um arquivo armazenado internamente com outro externo e dizer se são iguais (`diff arquivo.txt /tmp/outro.txt`)

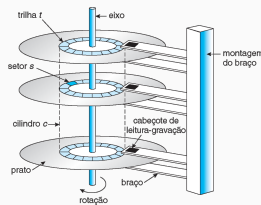
- duas estratégias possíveis para alocar um arquivo de n bytes
 - alocar os n bytes de maneira consecutiva
 - dividir o arquivo em uma série de blocos não necessariamente contíguos
- já vimos um cenário parecido em memória
 - diferença entre segmentação pura e paginação

- duas estratégias possíveis para alocar um arquivo de n bytes
 - alocar os n bytes de maneira consecutiva
 - problemas se o arquivo precisar crescer de tamanho → será necessário movê-lo
 - dividir o arquivo em uma série de blocos não necessariamente contíguos
- já vimos um cenário parecido em memória
 - diferença entre segmentação pura e paginação

- duas estratégias possíveis para alocar um arquivo de n bytes
 - alocar os n bytes de maneira consecutiva
 - problemas se o arquivo precisar crescer de tamanho → será necessário movê-lo
 - dividir o arquivo em uma série de blocos não necessariamente contíguos
 - esta acaba sendo a estratégia usada pela ampla maioria dos sistemas de arquivos
- já vimos um cenário parecido em memória
 - diferença entre segmentação pura e paginação
 - mas na memória, mover dados é muito mais rápido que no disco

Tamanho do bloco

- é preciso definir qual será o tamanho dos blocos usados no armazenamento
- se for muito grande (e.g., um cilindro inteiro)
 - qualquer arquivo vai ocupar pelo menos 1 cilindro (mesmo que só tenha 1 byte)
 - com arquivos pequenos haverá grande desperdício de espaço
- se for muito pequeno
 - a maioria dos arquivos precisará de múltiplos blocos
 - serão necessárias múltiplas buscas e atrasos rotacionais → piora o desempenho
- temos um dilema!
 - se for grande demais → desperdiçamos espaço
 - se for pequeno demais → desperdiçamos tempo



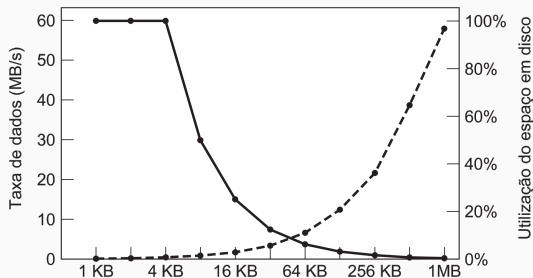
Fonte: [SGG15]

Tamanho do bloco

- o tamanho ideal depende da distribuição do tamanho dos arquivos armazenados
 - 4 KB é um tamanho bastante usado
- exemplo – disco com 1 MB por trilha
 - tempo de rotação de 8,33 ms (i.e., um disco de 7200 RPM)
 - atraso rotacional médio é $\frac{1}{2} \times 8,33 = 4,165$
 - tempo de busca de 5ms (necessário pra posicionar cabeçote na trilha correta)
 - tempo em ms pra ler um bloco de k bytes é $5 + 4,165 + (k/1000000) \times 8,33$
 - i.e., tempo de busca + atraso rotacional + transferência

Tamanho do bloco

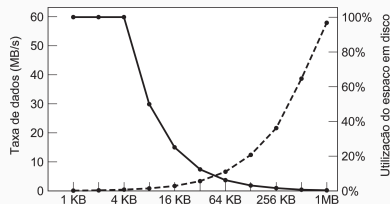
- taxa de dados vs tamanho do bloco para o disco do exemplo (tracejado)
- eficiência de espaço vs tamanho do bloco (curva sólida)
- tempo de acesso para 1 bloco é dominado pelo tempo de busca e atraso rotacional
 - $\approx 9\text{ms}$ pra acessar um bloco
 - quanto mais dados forem buscados, melhor!
- eficiência pra arq. de 4KB
 - com blocos de 1, 2, ou 4KB é de 100%
 - usa respec. 4, 2 ou 1 bloco
 - 50% c/ 8KB, 25% c/ 16KB, ...



obs: estamos assumindo que todos os arquivos tem tamanho de 4 KB

Tamanho do bloco

- na prática poucos arquivos tem tamanho múltiplo exato do tamanho do bloco
 - logo, sempre desperdiça algum espaço no último bloco
- as 2 curvas mostram desempenho e utilização do espaço em conflito
 - não há equilíbrio razoável
 - as curvas se cruzam quando o tamanho do bloco é 64 KB
 - taxa de dados é 6,6 MB/s
 - eficiência é de $\approx 7\%$
- historicamente os sistemas de arquivos usam tamanhos entre 1 e 4KB
 - talvez aumente com discos maiores (e.g., alguns TB)



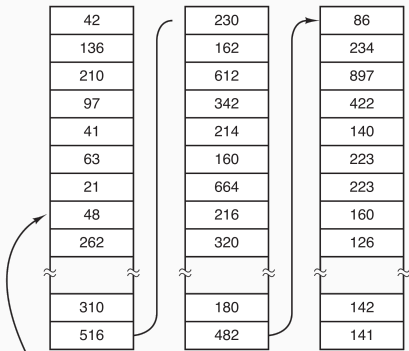
obs: estamos assumindo que todos os arquivos tem tamanho de 4 KB

Monitoramento dos blocos livres

- há dois métodos geralmente usados para controlar quais blocos estão livres/disponíveis
 1. lista encadeada de blocos de disco
 - cada bloco contém números de blocos livres
 - com bloco de 1 KB e número de bloco com 32 bits cabem os números de 255 blocos + 1 ponteiro pro próximo nó
 - disco de 1TB (≈ 1 bi de blocos), guardar todos exige 4 mi de blocos na lista encadeada (que pode ficar no espaço livre)
 2. mapa de *bits*
 - disco com n blocos exige um mapa com n bits
 - 0s e 1s são usados pra indicar livre/ocupado
 - disco de 1TB precisa de 1 bi de *bits* pro mapa $\rightarrow$ 130 mil blocos de 1KB
- mapa de *bits* usa 1 *bit* por bloco enquanto a lista usa 32
 - logo exige menos espaço
 - apenas quando o disco estiver quase cheio que a lista ocupará menos

Monitoramento dos blocos livres

Blocos livres de disco: 16, 17, 18



Um bloco de disco de 1 KB pode conter 256 números de blocos de disco de 32 bits

1001101101101100
0110110111110111
1010110110110110
0110110110111011
1110111011101111
1101101010001111
0000111011010111
1011101101101111
1100100011101111
~
0111011101110111
1101111101110111

Um mapa de bits

Monitoramento dos blocos livres

- se os blocos livres tendem a aparecer em (pequenos) grupos há uma otimização possível na lista encadeada
 - guardar conjuntos de blocos
 - e.g., contador de 8, 16 ou 32 *bits* informando o nº de blocos livres consecutivos
 - em um disco (quase) vazio, basta guardar dois números: endereço do 1º bloco livre + contador de blocos livres consecutivos
 - e se o disco estiver muito fragmentado?
 - aí é pior, pois guarda o contador além do número do endereço
- Sistema Operacional precisa tomar uma decisão cuja qualidade (melhor ou pior) depende do que acontecerá no futuro

Monitoramento dos blocos livres

- em cada momento, apenas um bloco de ponteiros para blocos livres precisa estar na memória
 - ao criar um arquivo, os (endereços dos) novos blocos necessários são tirados deste bloco
 - quando ele estiver vazio, basta ler um novo bloco do disco
 - ao liberar um arquivo, os endereços dos blocos são inseridos no bloco de ponteiros que está na RAM
 - se o bloco de ponteiros encher, ele será escrito no disco

Cotas de disco

- em ambiente multiusuário é preciso evitar exageros no uso do espaço compartilhado
 - motivação pra mecanismos de cota de disco
- administrador(a) do sistema designa uma cota
 - SO verifica periodicamente para que os usuários não a excedam
- exemplo – usuário abre um arquivo
 1. atributos e endereços de disco são localizados e colocados na memória (em uma tabela)
 2. um dos atributos guarda quem é o proprietário
 - alterações no tamanho do mesmo são computadas na cota do proprietário
 3. uma segunda tabela guarda dados das cotas dos usuários
 - limite flexível pode ser excedido (temporariamente)
 - limite estrito não pode ser excedido

Cotas de disco

- quando o usuário entra no sistema, o arquivo de cotas (com a tabela) é verificado
 - limite flexível tanto para o nº de arquivos ou blocos de disco
- usuário recebe um aviso caso tenha ultrapassado o limite
 - um contador de avisos restantes é decrementado
 - se chegar a 0, o usuário é bloqueado
- esta estratégia permite que os usuários ultrapassem os limites flexíveis enquanto estão logados
 - devem liberar antes do logout
 - limites estritos jamais podem ser excedidos

Tabela de arquivos abertos

Endereços em disco dos atributos Usuário = 8 Ponteiro da cota

Tabela de cotas

Limite flexível de blocos
Limite estrito de blocos
nº atual de blocos
nº de avisos restantes de uso de blocos
Limite flexível de arquivos
Limite estrito de arquivos
nº atual de arquivos
nº de avisos restantes de uso de arquivos

Registro de cota do usuário 8

BACKUPS (CÓPIAS DE SEGURANÇA) DO SISTEMA DE ARQUIVOS

- em muitos casos (maioria?), perder os arquivos é muito pior do que perder o computador!
 - substituir um computador custa \$\$, mas dá pra fazer sem tanto esforço
 - especialmente PCs
 - recuperar as informações pode ser muito caro e muito difícil
 - muitas vezes será impossível
- o sistema de arquivos não consegue oferecer proteção contra destruição física dos equipamentos e das mídias
 - mas pode ajudar a proteger as informações
- solução é fazer cópias de segurança (*backups*)
 - parece simples, mas nem sempre é...

BACKUPS (CÓPIAS DE SEGURANÇA) DO SISTEMA DE ARQUIVOS

- *backups* são úteis para dois problemas potenciais
 - recuperação em caso de um desastre
 - e.g., quebra de disco, enchente, incêndio, ciclone, etc
 - recuperação de uma bobagem feita
 - e.g., remoção acidental de arquivos
- fazer *backup* é demorado e ocupa bastante espaço
 - motivação pra fazer de maneira eficiente e conveniente
- algumas questões
 - o que deve ser copiado?
 - binários, temporários, arquivos especiais de E/S, etc não precisam ser copiados
 - o que mudou desde a última cópia?
 - ideia de cópias incrementais (cópias completas periódicas são intercaladas com algumas cópias incrementais)
 - torna a recuperação um pouco mais difícil
 - como lidar com grandes volumes?
 - compressão ajuda mas torna menos confiável em caso de falhas no *backup*
 - *backup* com o sistema rodando

- também devem ser considerados aspectos não técnicos
 - e.g., segurança do sistema vs segurança física do local onde ficam as mídias de *backup*
 - roubo
 - incêndio, etc
- estratégias principais
 - **cópia física**
 - escreve todos os blocos do disco de entrada no disco de saída
 - **cópia lógica**
 - copia os arquivos e diretórios selecionados

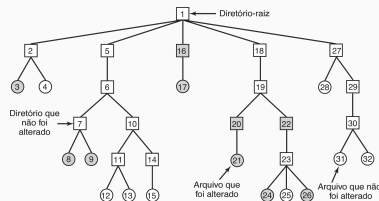
Cópia física

- copiar 100% vai incluir tudo
 - inclusive os blocos que não estão sendo usados
 - é possível pular estes blocos (após identificar) com alguma complexidade adicional
 - e.g., escrever o número de cada bloco antes de escrevê-lo (já que o bloco k do *backup* não é mais necessariamente o bloco k do disco)
 - blocos defeituosos não devem ser copiados
 - sempre há blocos defeituosos (alguns o HW do disco consegue ocultar)
 - os que não forem ocultados podem estar listados em um arquivo de blocos defeituosos → não faz sentido copiar
 - arquivos de paginação e hibernação do Windows
- principais vantagens: velocidade e simplicidade
- desvantagens: incapacidade de pular diretórios / de fazer cópias incrementais / de restaurar arquivos específicos

BACKUPS (CÓPIAS DE SEGURANÇA) DO SISTEMA DE ARQUIVOS

Cópia lógica

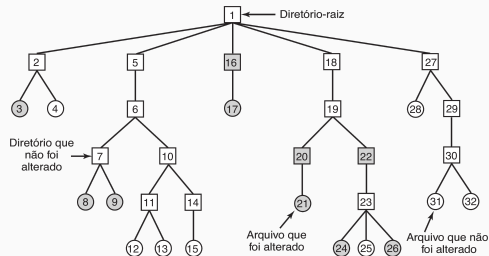
- é a solução adotada na maioria dos casos
- permite especificar arquivos e diretórios a copiar (ou pular)
- permite copiar apenas aqueles modificados a partir de uma data base
- exemplo – algoritmo usado em sistemas UNIX
- arquivos (círculos) e diretórios (quadrados) em cinza foram modificados desde a data base
 - os demais arquivos não serão copiados
 - os diretórios sempre são copiados
 - para facilitar a restauração
 - para permitir a restauração incremental
- o algoritmo usa um mapa de *bits* indexado pelo nº do *i-node*
 - conforme o algoritmo executa os *bits* são alterados para 1 ou 0



BACKUPS (CÓPIAS DE SEGURANÇA) DO SISTEMA DE ARQUIVOS

Cópia lógica

- algoritmo possui 4 fases
 1. começa no diretório inicial
 - para cada arquivo modificado, seu *i-node* é marcado no mapa
 - assim como todos os diretórios
 2. percorre a árvore novamente desmarcando diretórios que não tenham arquivos/diretórios modificados (recursivamente)
 - desmarcou diretórios 10, 11, 14, 27, 29 e 30
 - 5 e 6 serão copiados pois são necessários para restaurar mudanças recentes
 - a partir daqui, sabemos quais arquivos e diretórios copiar

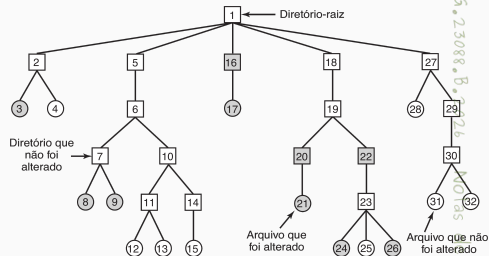


1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32
---	---	---	---	---	---	---	---	---	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----

BACKUPS (CÓPIAS DE SEGURANÇA) DO SISTEMA DE ARQUIVOS

Cópia lógica

- algoritmo possui 4 fases
 3. escaneia os *i-nodes* e ordem numérica e copia todos os diretórios marcados (ver fig.)
 - todos são prefixados de seus atributos (proprietário, horários, etc)
 4. copia os arquivos marcados (ver fig.)
 - assim como os diretórios, são prefixados de seus atributos (proprietário, horários, etc)
 - fim da cópia



1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32
---	---	---	---	---	---	---	---	---	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----

Cópia lógica

- restaurar os arquivos é relativamente simples
 1. cria um sistema de arquivos vazio no novo disco
 2. restaura a cópia completa mais recente
 - os diretórios estarão presentes, então teremos um esqueleto do sistema de arq.
 - os arquivos são restaurados
 3. o processo se repete com as cópias incrementais
- há algumas complicações
 - é preciso criar a lista de blocos livres
 - ligações (*links*)
 - evitar restaurar o mesmo arquivo 2×
 - arquivos com lacunas (e.g., core dump do UNIX)
 - se preencher as lacunas com 0, o arquivo terá o tamanho do espaço de endereçamento!

CONSISTÊNCIA DO SISTEMA DE ARQUIVOS

v.FURG.230xx.B.2024 - Notas de Aula para Discintos (36)

- quedas podem tornar o sistema de arquivos inconsistente
 - especialmente se ocorrerem no meio da escrita de *i-nodes*, diretórios ou blocos da lista de blocos livres
- SOs incluem utilitários para conferir a consistência
 - e.g., fsck do UNIX
 - e.g., sfc do Windows
 - em geral executados sempre que o sistema reinicia após uma parada abrupta
 - se baseiam na redundância do sistema de arquivos para fazer reparos
- verificação pode ser de blocos e de arquivos
- p/ conferir blocos, são usadas 2 tabelas com contadores para cada bloco
 - a primeira tabela monitora qtas vezes o bloco está presente em um arquivo
 - a segunda registra quantas vezes cada bloco aparece entre os blocos livres
 - programa lê os *i-nodes* em modo cru (*raw*) retornando todos os blocos de disco a partir de 0

CONSISTÊNCIA DO SISTEMA DE ARQUIVOS

- para conferir blocos, são usadas 2 tabelas com contadores para cada bloco
 - a partir de um *i-node* é possível construir a lista de todos os números de blocos usados no arquivo em questão
 - ao ler cada número de bloco, o contador na primeira tabela é incrementado
 - o programa examina os blocos livres pra encontrar os que não estão sendo usados
 - incrementa o contador na segunda tabela
 - se o sistema estiver consistente, cada bloco terá 1 apenas na primeira ou apenas na segunda tabela

Número do bloco

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15

1	1	0	1	0	1	1	1	1	0	0	1	1	1	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Blocos em uso

0	0	1	0	1	0	0	0	0	1	1	0	0	0	1	1
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Blocos livres

CONSISTÊNCIA DO SISTEMA DE ARQUIVOS

- p/ conferir blocos, são usadas 2 tabelas com contadores para cada bloco
 - em caso de queda, podemos ter outro cenário
 - bloco 2 não ocorre em nenhuma → bloco desaparecido

Número do bloco

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15

1	1	0	1	0	1	1	1	1	0	0	1	1	1	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Blocos em uso

0	0	0	0	1	0	0	0	0	1	1	0	0	0	1	1
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Blocos livres

- não é um prejuízo real mas desperdiça espaço do disco
- ao recuperar, o verificado o inclui nos blocos livres

CONSISTÊNCIA DO SISTEMA DE ARQUIVOS

- p/ conferir blocos, são usadas 2 tabelas com contadores para cada bloco
 - também pode ocorrer de um bloco (4) estar duas vezes entre os livres
 - só é possível com lista encadeada (jamais com mapa de bits)
 - solução é reconstruir a lista de livres

Número do bloco

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15

1	1	0	1	0	1	1	1	1	0	0	1	1	1	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Blocos em uso

0	0	1	0	2	0	0	0	0	1	1	0	0	0	1	1
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Blocos livres

CONSISTÊNCIA DO SISTEMA DE ARQUIVOS

- p/ conferir blocos, são usadas 2 tabelas com contadores para cada bloco
 - situação pior é um bloco (5) estar em dois ou mais arquivos
 - se um deles for removido, o bloco será liberado → estará ao mesmo tempo em uso e livre
 - se os dois forem removidos → aparecerá 2× entre os livres
 - solução é alocar um novo bloco, copiar o conteúdo e deixar cada arquivo com a sua cópia
 - o conteúdo de ambos fica inalterado (embora provavelmente adulterado)
 - a situação deve ser reportada p/ que o usuário inspecione o dano

Número do bloco

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15

1	1	0	1	0	2	1	1	1	0	0	1	1	1	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Blocos em uso

0	0	1	0	1	0	0	0	0	1	1	0	0	0	1	1
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Blocos livres

- após conferir a contabilização de cada bloco, passa-se p/ a verificação do sistema de diretórios
- usa uma tabela de contadores com entradas p/ cada arquivo
- começa na raiz e desce recursivamente
- p/ cada *i-node* em cada diretório, incrementa um contador representando o uso do arquivo
 - cada arquivo pode aparecer em mais de um diretório caso existam ligações estritas
 - ligações simbólicas não interferem aqui
- ao final, haverá uma tabela indexada pelo nº do *i-node* dizendo quantos diretórios contém cada arquivo
 - tal nº é comparado com o contador presente no *i-node*
 - ambas as contagem devem conter o mesmo valor

- 2 erros possíveis
 - contagem no *i-node* é alta demais
 - se for mais alta que o nº presente nos diretórios, evitará que o *i-node* seja removido mesmo que todas as entradas sejam excluídas
 - não é grave → apenas inutiliza parte do espaço do disco
 - solução é atribuir o valor correto ao contador do *i-node*
 - contagem no *i-node* é baixa demais
 - erro potencialmente catastrófico
 - ao remover qualquer uma das entradas, o *i-node* será removido (qdo chegar a 0)
→ sistema libera todos os blocos do arquivo
 - teremos um diretório que aponta pra um *i-node* não usado cujos blocos podem ser atribuídos a outros arquivos
 - solução é forçar o contador do *i-node* a assumir o valor correto

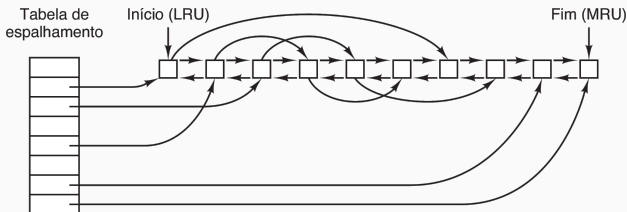
- acessar o disco é muito mais lento do que acessar a memória
 - ler 32 *bits* da memória pode levar 10 ns
 - o disco pode ser lido a 100 MB/s (4× mais lento)
 - e ainda é preciso somar 5-10ms p/ buscar a trilha e esperar o setor desejado passar pelo cabeçote de leitura
 - se o volume acessado for pequeno, acessar a memória pode ser em torno de 1 milhão de vezes mais rápido!
 - motivação para algumas motivações

Cache de blocos

- técnica comum é usar uma cache de blocos ou cache de buffer
 - uma coleção de blocos que pertencem ao disco mas que são mantidos temporariamente na memória
- na solicitação de um bloco, 1º ele é buscado na cache
 - se estiver, a leitura é feita sem acessar o disco
 - se não estiver, ele é lido do disco p/ a cache e depois para onde foi solicitado
 - solicitações seguintes ao mesmo arquivo serão atendidas a partir da cache

Cache de blocos

- há potencialmente muitos blocos na cache
 - é preciso uma maneira rápida p/ verificar se um dado bloco está ou não presente
 - e.g., uma tabela *hash*



- quando a cache está cheia, algum algoritmo de substituição é necessário
 - e.g., LRU
 - blocos podem ser mantidos em uma lista (duplamente) encadeada com ponteiros p/ LRU e MRU

Cache de blocos

- mas há um problema em usar o LRU
 - aparece em caso de quedas no sistema
 - e pode levar a problemas de consistência
 - exemplo – um bloco crítico (e.g., *i-node*) é modificado na cache
 - não será escrito no disco imediatamente, apenas quando se tornar o primeiro (LRU)
 - se o sistema cair antes disso → ficará em estado inconsistente
 - além disso, *i-nodes* raramente são referenciados duas vezes em um intervalo curto
 - motivação para um LRU modificado

Cache de blocos

- LRU modificado
 1. É provável que o bloco seja necessário logo novamente?
 - caso não seja, vai pro início da fila e não para o fim → será removido mais rapidamente
 2. O bloco é essencial para a consistência do sistema de arquivos?
 - neste caso deve ser escrito em disco imediatamente
- além disso, é indesejável manter (quaisquer) blocos de dados em cache por muito tempo
 - risco de perder do usuário perder o trabalho (ainda que o sistema em si esteja consistente)
 - há duas abordagens possíveis
 1. UNIX – chamada de sistema sync executada de tempos em tempos (e.g., 30s)
 2. Windows – cache de escrita direta (*write-through caches*) que escreve o bloco pro disco assim que for modificado em cache

Leitura antecipada de blocos

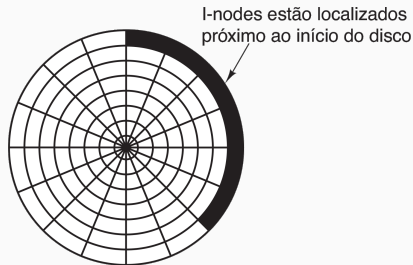
- uma outra otimização possível é tentar trazer os dados $p/$ a cache antes que sejam solicitados
 - visa aumentar a taxa de acertos
- é útil pra arquivos lidos sequencialmente
 - quando o sistema atende uma solicitação por um bloco k , ele confere a cache buscando por $k + 1$ e se necessário já o busca
- é ruim pra leituras aleatórias
 - ocupa a largura de banda do disco inutilmente atrasando as leituras úteis
- uma solução é o sistema monitorar $p/$ tentar identificar se vale a pena ou não fazer a antecipação
 - para cada arquivo aberto, de início coloca em acesso sequencial
 - se as leituras forem sequenciais, mantém
 - caso o arquivo faça *seek*, ele passa a considerar como leituras aleatórias

Redução do movimento do braço do disco

- reduzir os movimentos necessários do braço do disco também pode ajudar no desempenho
- e.g., colocando próximos blocos que têm maior chance de serem acessados em sequência
 - ao escrever um arquivo, com um mapa de *bits* é fácil escolher o bloco livre mais próximo do bloco atual
 - já com lista encadeada de blocos livres é muito mais difícil
 - ainda assim é possível se consideramos grupos de blocos consecutivos e não blocos individuais
- sistemas que usam *i-nodes* precisam de (pelo menos) dois acessos pra ler um arquivo: um pro *i-node* e outro pro bloco

Redução do movimento do braço do disco

- se os *i-nodes* estão no início e os blocos mais pra frente, a distância vai exigir buscar longas



- uma melhoria é colocar os *i-nodes* no meio do disco, reduzindo a distância média de busca
 - ou então dividir o disco em grupos de cilindros, cada grupo com seus *i-nodes*, blocos e lista de livres
 - ao escrever, tenta colocar os blocos novos no mesmo grupo



- quando o SO é instalado, os programas e arquivos que ele precisa são instalados de modo consecutivo
 - i.e., começa no início do disco, e um segue o outro
 - logo, todo espaço livre está no final do disco (em área contígua)
- ao longo do tempo de operação, arquivos são criados e removidos
 - disco se torna fragmentado!
 - i.e., mescla de arquivos com espaços vazios
- ao criar um novo arquivo, é preciso buscar por um espaço pra ele
 - agora este espaço pode estar em qualquer lugar → desempenho ruim
- solução é mover os arquivos para que fiquem contíguos
 - assim como espaço livre

DESFRAAGMENTAÇÃO DE DISCO

- para funcionar melhor, a desfragmentação precisa de “bastante” espaço livre em uma região contígua (no fim da partição)
 - espaço necessário pra que o programa de desfragmentação funcione
 - copia os arquivos fragmentados do início pro final
 - copia de volta do final pro início, deixando-os de maneira contígua
- sistemas de arquivos Linux (e.g., ext3), em geral, sofrem menos com fragmentação
 - consequência da maneira como os blocos de disco são selecionados
 - raramente é necessário invocar a desfragmentação manual

- para funcionar melhor, a desfragmentação precisa de “bastante” espaço livre em uma região contígua (no fim da partição)
 - espaço necessário pra que o programa de desfragmentação funcione
 - copia os arquivos fragmentados do início pro final
 - copia de volta do final pro início, deixando-os de maneira contígua
- sistemas de arquivos Linux (e.g., ext3), em geral, sofrem menos com fragmentação
 - consequência da maneira como os blocos de disco são selecionados
 - raramente é necessário invocar a desfragmentação manual
- já sistemas Windows são mais afetados
- atenção: SSDs não devem ser desfragmentados
 - não há ganho de desempenho
 - escritas desnecessárias → desgaste